

UNIVERSITÀ DEGLI STUDI DI MILANO-BICOCCA

DIPARTIMENTO DI INFORMATICA, SISTEMISTICA E
COMUNICAZIONE

CORSO DI LAUREA MAGISTRALE IN INFORMATICA



Architettura Dati

Valutazione dell'Impatto del Rumore sulla Predizione dei
Modelli di Machine Learning

Autore:

Alen Bicanic Matr. 945230

Samuele Perotti Matr. 899817

Anno Accademico 2025-2026

Indice

1	Introduzione	2
2	Descrizione del dataset	3
3	Preprocessing dei dati	5
3.1	Rimozione di colonne non informative	5
3.2	Conversione del target in problema binario	5
3.3	Gestione delle variabili booleane	5
3.4	One-hot encoding delle variabili categoriche	5
3.5	Gestione dei valori mancanti residui	5
3.6	Scaling delle feature	6
4	Analisi esplorativa (EDA)	7
4.1	Distribuzione del target binario	7
4.2	Matrice di correlazione	8
4.3	Distribuzione delle variabili numeriche	9
5	Suddivisione train/test	10
6	Modelli di baseline	10
6.1	Decision Tree	10
6.1.1	Decision Tree - Pruned	11
6.2	Neural Network (Multi-Layer Perceptron)	11
6.3	Support Vector Machine (SVM)	14
6.4	Confronto tra i modelli di baseline	14
7	Introduzione del rumore nel dataset	16
7.1	Valori mancanti (NaN)	16
7.2	Valori duplicati	16
7.3	Dati "sporchi"	16
8	Impatto del rumore sulle prestazioni dei modelli	18
8.1	Valori mancanti	18
8.1.1	Cross-Validation	22
8.2	Valori duplicati	23
8.2.1	Cross-Validation	26
8.3	Dati sporchi	27
8.3.1	Cross Validation	30
9	Tecniche di preprocessing e regolarizzazione per la mitigazione del rumore	32
9.1	Mitigazione dei valori mancanti	32
9.2	Mitigazione dei valori duplicati	33
9.3	Mitigazione dei dati sporchi	34
10	Confronto finale tra le strategie	36
11	Conclusioni	36

1 Introduzione

L'obiettivo del progetto è comprendere in che modo la presenza di rumore nei dati influenzi le prestazioni dei modelli di Machine Learning e individuare le strategie più efficaci per limitarne l'impatto. Il lavoro si è articolato nelle seguenti fasi, in linea con la proposta progettuale concordata:

- scelta e analisi di un dataset adatto al problema di classificazione;
- preparazione dei dati e creazione del training set;
- introduzione di rumore nel dataset a diversi livelli di intensità;
- addestramento di più modelli sul dataset modificato e analisi delle prestazioni al variare del rumore;
- applicazione di tecniche di preprocessing e regolarizzazione (normalizzazione/scaling, PCA, cross-validation, early stopping, dropout, imputazione avanzata, gestione dei duplicati) per valutarne l'efficacia nel contenere il degrado delle prestazioni;
- confronto tra i risultati ottenuti sui dataset rumorosi e quelli ottenuti sul dataset originale;
- valutazione delle performance mediante le metriche di accuracy, precision e recall.

Il dominio applicativo scelto è quello medico/di supporto alla diagnosi: si è utilizzato l'*UCI Heart Disease Dataset*, con l'obiettivo di prevedere la presenza o assenza di una malattia cardiaca a partire da variabili cliniche (classificazione binaria).

2 Descrizione del dataset

Il dataset utilizzato è lo **UCI Heart Disease Dataset**, reso disponibile su GitHub¹ e caricato in questo progetto da un repository GitHub del corso. Il dataset è composto da **920 istanze** e, nella sua forma originale, da **16 colonne**, tra cui variabili demografiche (età, sesso), cliniche (pressione a riposo, colesterolo, frequenza cardiaca massima, angina da sforzo, ecc.) e la variabile target **num**.

La variabile target originale è **multiclasse** (valori da 0 a 4, dove 0 indica assenza di malattia e i valori da 1 a 4 indicano livelli crescenti di gravità). La distribuzione originale delle classi è riportata in Tabella 1.

Classe (num)	Numero di istanze
0 (assenza)	411
1	265
2	109
3	107
4	28

Tabella 1: Distribuzione originale della variabile target multiclasse.

Diverse variabili presentano valori mancanti in misura significativa (ad esempio la colonna **ca**, presente solo in 309 istanze su 920), aspetto che ha reso necessaria un'attenta fase di preprocessing, descritta nella sezione seguente.

- **id**: identificatore univoco del paziente.
- **age**: età del paziente espressa in anni.
- **origin (dataset)**: luogo di provenienza dello studio clinico.
- **sex**: sesso del paziente (Male/Female).
- **cp (chest pain type)**: tipologia di dolore toracico, con le seguenti modalità:
 - typical angina
 - atypical angina
 - non-anginal pain
 - asymptomatic
- **trestbps**: pressione arteriosa a riposo (in mm Hg) misurata al momento del ricovero.
- **chol**: livello di colesterolo (mg/dl).
- **fbs (fasting blood sugar)**: indica se la glicemia a digiuno è superiore a 120 mg/dl (True/False).

¹https://raw.githubusercontent.com/Bicanic-Alen/Progetto_Archittura_Dati_2026/refs/heads/master/heart_disease_uci.csv

- **restecg**: risultati dell'elettrocardiogramma a riposo:
 - normal
 - ST-T wave abnormality
 - left ventricular hypertrophy
- **thalach (maximum heart rate achieved)**: frequenza cardiaca massima raggiunta durante il test.
- **exang**: presenza di angina indotta dall'esercizio fisico (True/False).
- **oldpeak**: depressione del tratto ST indotta dall'esercizio rispetto alla condizione di riposo.
- **slope**: pendenza del segmento ST al picco dell'esercizio.
- **ca**: numero di vasi principali (0–3) colorati tramite fluoroscopia.
- **thal**: risultato del test al tallio:
 - normal
 - fixed defect
 - reversible defect
- **num**: variabile target originale, che indica la presenza e la severità della malattia cardiaca.

3 Preprocessing dei dati

Prima dell'addestramento dei modelli, il dataset è stato sottoposto alle seguenti operazioni di pulizia e trasformazione.

3.1 Rimozione di colonne non informative

Le colonne `id` (identificativo univoco della riga) e `dataset` (indicante solo la provenienza geografica del campione) sono state rimosse in quanto non contribuiscono alla predizione.

3.2 Conversione del target in problema binario

Per semplicità e coerenza clinica, il target multiclasse `num` è stato trasformato in una variabile binaria `target`:

- 0 = assenza di malattia;
- 1 = presenza di malattia (aggregazione delle classi originarie 1–4).

3.3 Gestione delle variabili booleane

Le variabili `fbs` (glicemia a digiuno) ed `exang` (angina indotta da sforzo) sono state convertite in formato numerico 0/1, imputando i valori mancanti con `False` prima della conversione.

3.4 One-hot encoding delle variabili categoriche

Le variabili categoriche `sex`, `cp`, `restecg`, `slope` e `thal` sono state trasformate tramite *one-hot encoding* (con rimozione della prima categoria per evitare collinearità, `drop_first=True`).

3.5 Gestione dei valori mancanti residui

Dopo l'encoding, le colonne numeriche che presentavano ancora valori mancanti (`trestbps`, `chol`, `thalch`, `oldpeak`, `ca`) sono state imputate con la **mediana** della rispettiva colonna. Questa scelta è stata preferita alla media in quanto più robusta rispetto agli outlier e permette di non perdere istanze, aspetto particolarmente rilevante data la dimensione limitata del dataset. Il conteggio dei valori mancanti per colonna, riscontrato dopo l'encoding, è riportato in Tabella 2.

Colonna	Valori mancanti
<code>trestbps</code>	59
<code>chol</code>	30
<code>thalch</code>	55
<code>oldpeak</code>	62
<code>ca</code>	611

Tabella 2: Valori mancanti per colonna numerica prima dell'imputazione.

3.6 Scaling delle feature

Le variabili numeriche sono state normalizzate/scalate (StandardScaler) per garantire che feature con scale di grandezza diverse (es. colesterolo vs età) non sbilanciasero il peso durante l'addestramento, passaggio essenziale soprattutto per la Rete Neurale.

4 Analisi esplorativa (EDA)

Dopo il preprocessing è stata condotta un'analisi esplorativa per comprendere la struttura del dataset ripulito.

4.1 Distribuzione del target binario

Il grafico a barre della distribuzione del target binario mostra un discreto bilanciamento tra le due classi (assenza/presenza di malattia), condizione favorevole per l'addestramento dei modelli di classificazione.

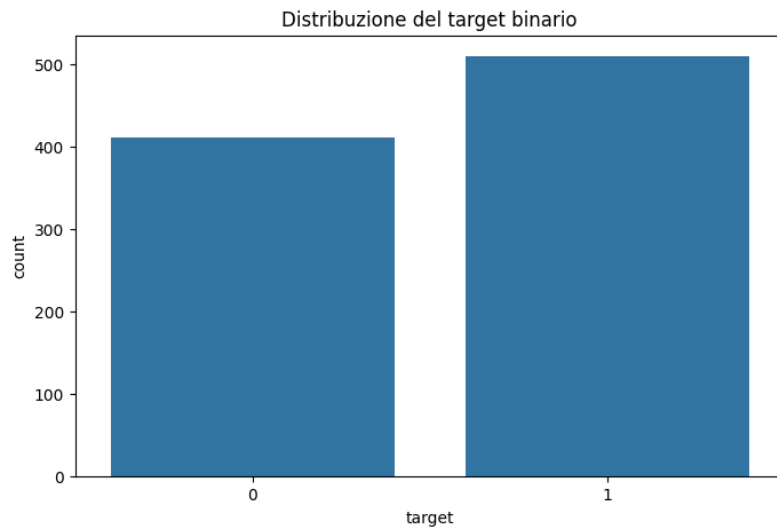


Figura 1: Distribuzione della variabile target binaria.

4.2 Matrice di correlazione

La matrice di correlazione tra tutte le variabili numeriche del dataset permette di individuare eventuali relazioni lineari forti tra le feature e con il target.

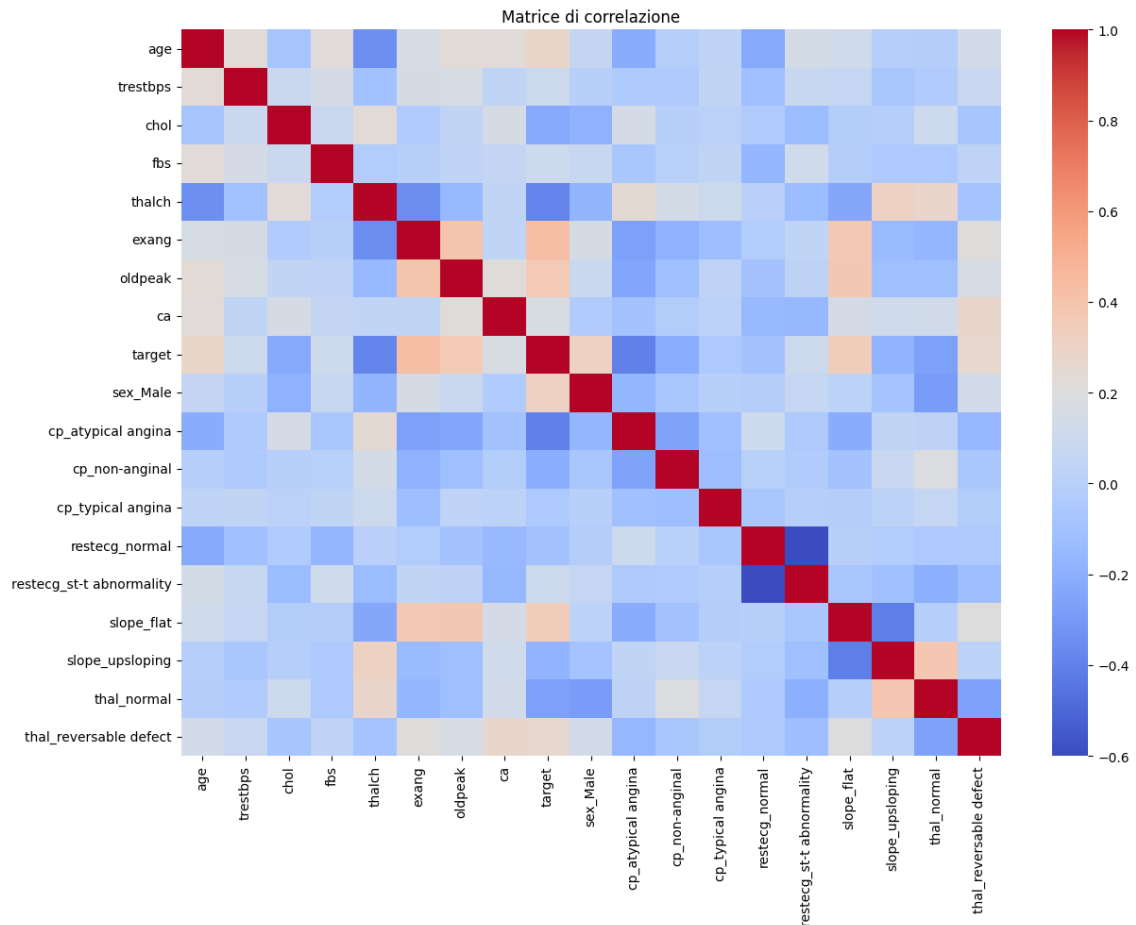


Figura 2: Matrice di correlazione tra le variabili del dataset

Dall'analisi esplorativa dei dati riportata nel notebook, è emerso attraverso la matrice di correlazione che variabili come il tipo di dolore toracico (cp) e la frequenza cardiaca massima (thalach) mostrano una correlazione positiva con la presenza della malattia. Al contrario, variabili come oldpeak mostrano una correlazione negativa. Inoltre, i grafici di distribuzione hanno evidenziato come l'età sia un fattore di rischio distribuito prevalentemente nella fascia over-50.

4.3 Distribuzione delle variabili numeriche

Gli istogrammi delle variabili numeriche permettono di valutarne la forma distributiva e la presenza di eventuali asimmetrie o outlier.

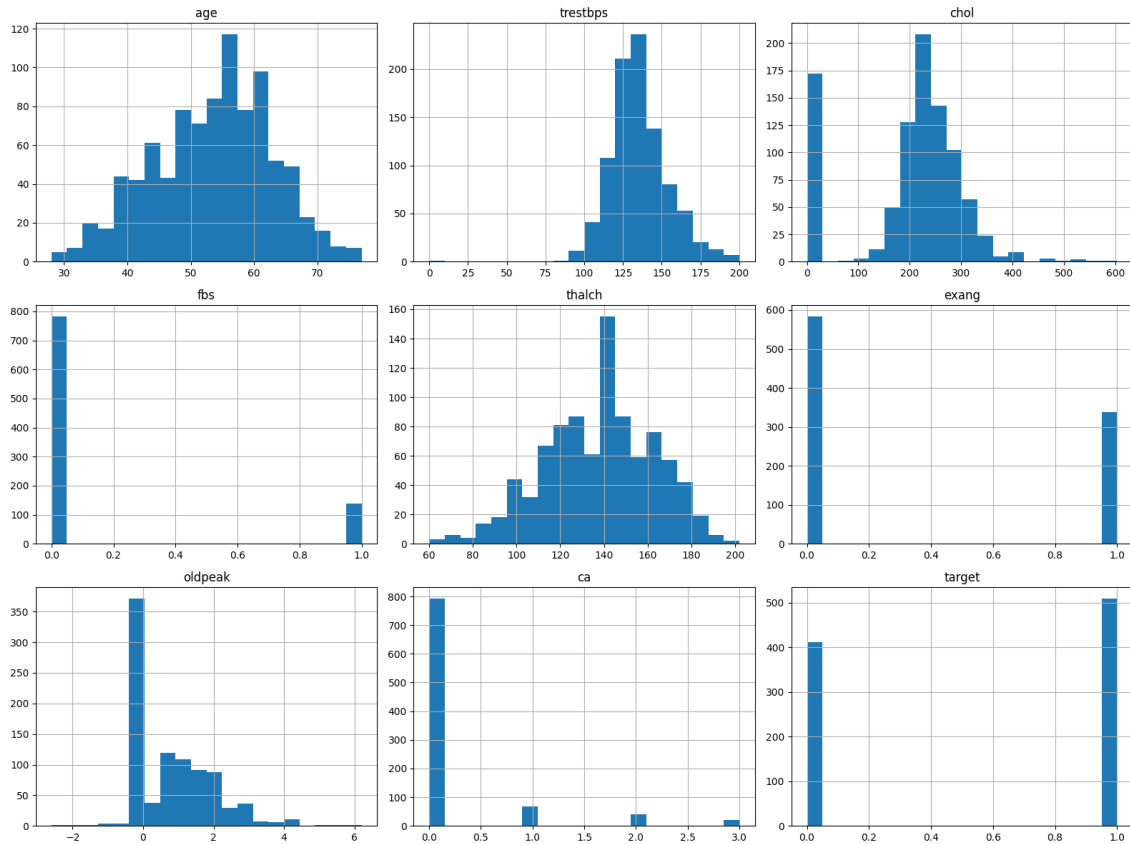


Figura 3: Istogrammi delle variabili numeriche

Gli istogrammi delle variabili numeriche mostrano distribuzioni non gaussiane e la presenza di asimmetrie, rafforzando la scelta della mediana come strategia di imputazione. I risultati dell'analisi esplorativa hanno fornito indicazioni fondamentali per le successive scelte di preprocessing, in particolare per la gestione dei valori mancanti, la codifica delle variabili categoriche e l'applicazione dello scaling delle feature.

5 Suddivisione train/test

Il dataset è stato suddiviso in training set e test set con proporzione 80/20 (`test_size=0.2`), utilizzando la stratificazione sulla variabile target (`stratify=y`) per preservare le proporzioni delle classi in entrambi i sottoinsiemi, e fissando `random_state=42` per la riproducibilità dei risultati.

6 Modelli di baseline

Sono stati addestrati e confrontati tre modelli di classificazione supervisionata sul dataset originale (privo di rumore aggiuntivo).

6.1 Decision Tree

È stato inizialmente addestrato un Decision Tree con criterio *entropy* senza vincoli di profondità, ottenendo i seguenti risultati sul test set:

Metrica	Classe 0	Classe 1	Media pesata
Precision	0.73	0.77	0.75
Recall	0.71	0.79	0.75
F1-score	0.72	0.78	0.75

Tabella 3: Decision Tree (non potato) – Accuracy sul test set: 0.7554

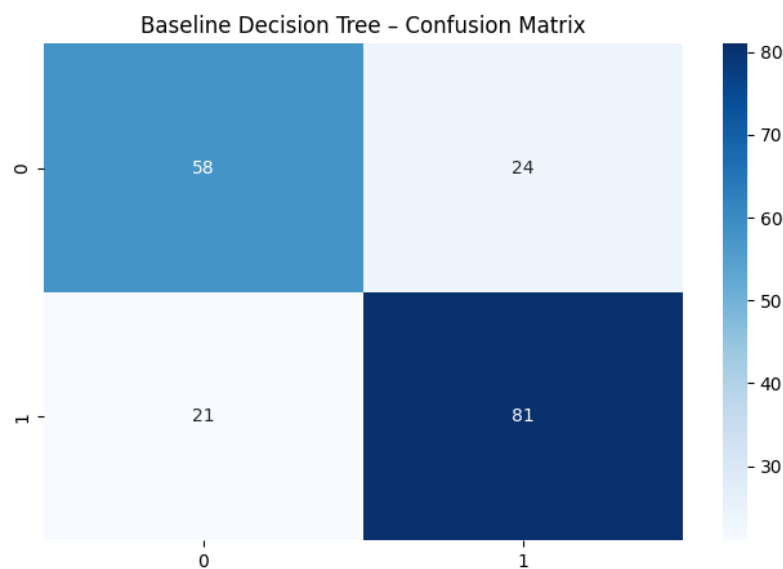


Figura 4: Matrice di confusione – Decision Tree (baseline)

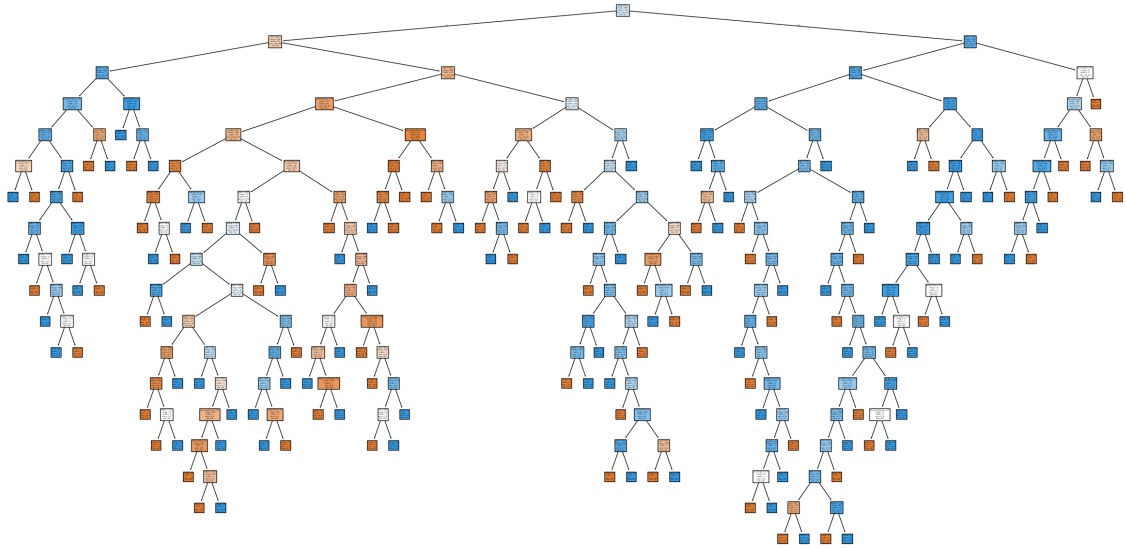


Figura 5: Albero di decisione non potato

6.1.1 Decision Tree - Pruned

L'albero non potato tende a sovradattare (*overfitting*) i dati di training. Per contenere questo effetto è stato introdotto un vincolo sulla profondità massima (`max_depth=5`), ottenendo un modello più semplice, più interpretabile e con prestazioni migliori sul test set:

Metrica	Classe 0	Classe 1	Media pesata
Precision	0.80	0.81	0.80
Recall	0.74	0.85	0.80
F1-score	0.77	0.83	0.80

Tabella 4: Decision Tree potato (`max_depth=5`) – Accuracy sul test set: 0.8043

La validazione tramite 5-fold cross-validation sul Decision Tree potato ha restituito un'accuracy media pari a **0.725**, coerente con quanto osservato sul singolo test set.

6.2 Neural Network (Multi-Layer Perceptron)

È stata definita una rete neurale feed-forward per la classificazione binaria, con la seguente architettura:

- Strato di input + primo hidden layer: 16 neuroni, attivazione sigmoide;
- Secondo hidden layer: 8 neuroni, attivazione sigmoide;
- Strato di output: 1 neurone, attivazione sigmoide (adatta alla classificazione binaria).

Il modello è stato addestrato per 100 epoche con `batch_size=16` sui dati precedentemente standardizzati (`StandardScaler`), passaggio necessario poiché le reti neurali sono sensibili alla scala delle feature in input.

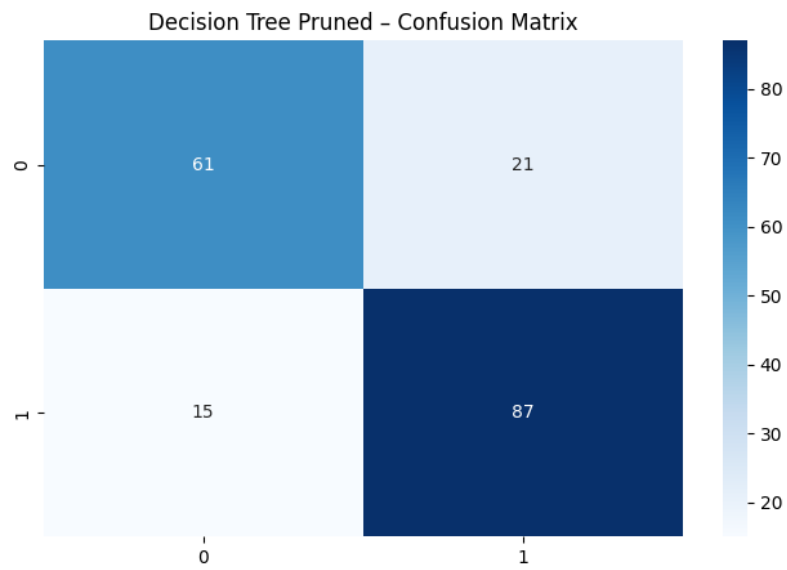


Figura 6: Matrice di confusione – Decision Tree Potato

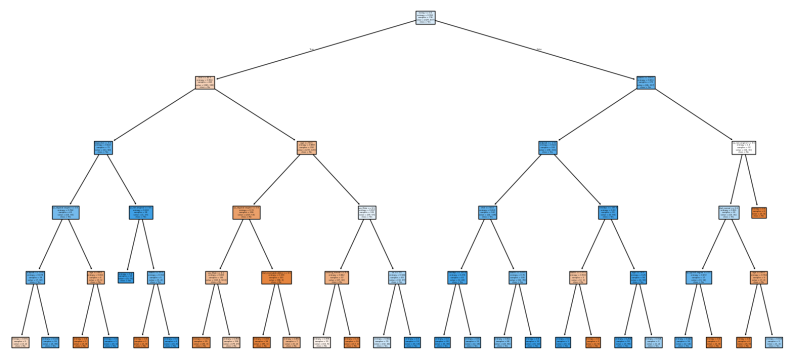


Figura 7: Matrice di confusione – Decision Tree Potato

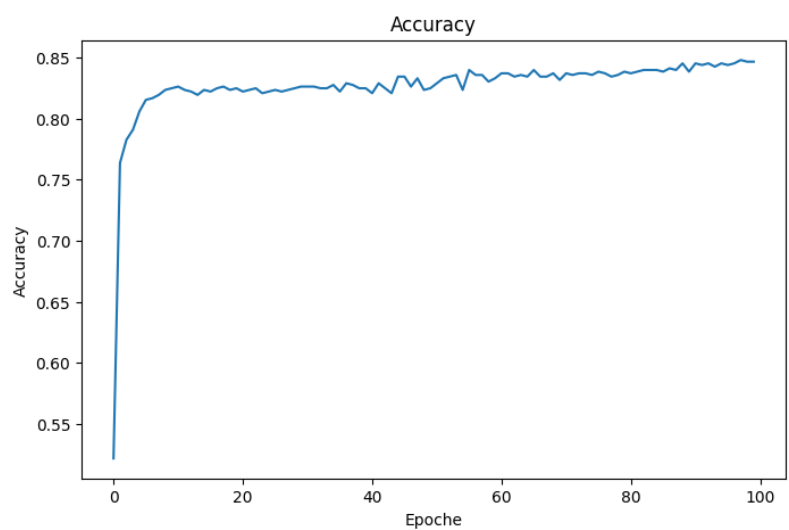


Figura 8: Andamento dell'accuracy di training (Neural Network)

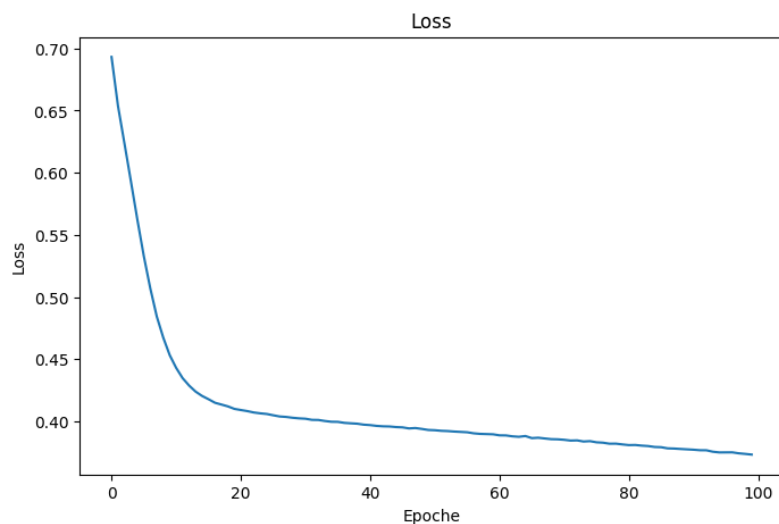


Figura 9: Andamento della loss di training (Neural Network)

Sul test set, il modello ha ottenuto una **Test Accuracy di 0.8315** (Test Loss = 0.3813), con le seguenti metriche di classificazione:

Metrica	Classe 0	Classe 1	Media pesata
Precision	0.83	0.83	0.83
Recall	0.78	0.87	0.83
F1-score	0.81	0.85	0.83

Tabella 5: Neural Network – Accuracy sul test set: 0.8315

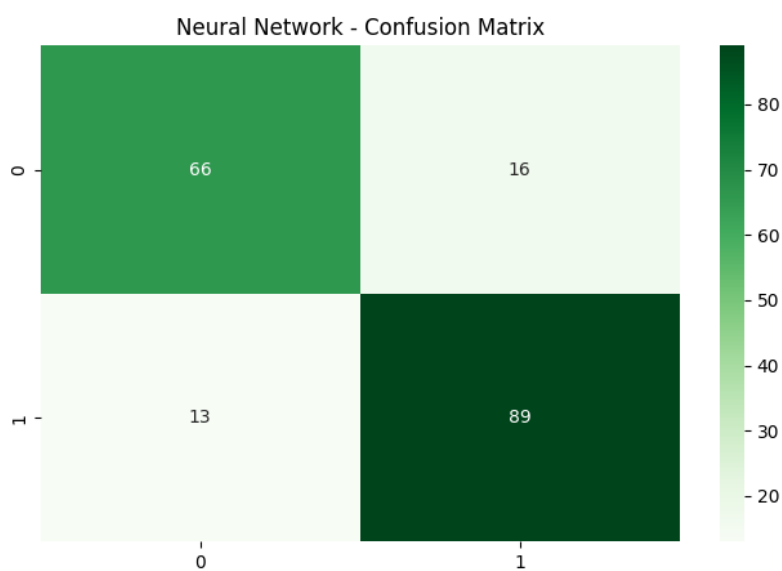


Figura 10: Matrice di confusione - Neural Network

6.3 Support Vector Machine (SVM)

Per il terzo modello è stata scelta una SVM con le seguenti motivazioni:

- **Kernel RBF (Radial Basis Function)**: a differenza di un kernel lineare, il kernel RBF gestisce relazioni non lineari tra le feature; dato che le variabili cliniche (pressione, colesterolo, età, ecc.) spesso interagiscono in modo complesso, un confine di decisione non lineare tende a offrire prestazioni migliori;
- **Parametro di regolarizzazione $C = 1.0$** : controlla il compromesso (*trade-off*) tra l'ampiezza del margine di separazione e la corretta classificazione dei punti di training.

Il modello, addestrato sui dati standardizzati, ha ottenuto un'Accuracy di **0.8424** sul test set:

Metrica	Classe 0	Classe 1	Media pesata
Precision	0.90	0.81	0.85
Recall	0.73	0.93	0.84
F1-score	0.81	0.87	0.84

Tabella 6: SVM (kernel RBF, $C=1.0$) – Accuracy sul test set: 0.8424

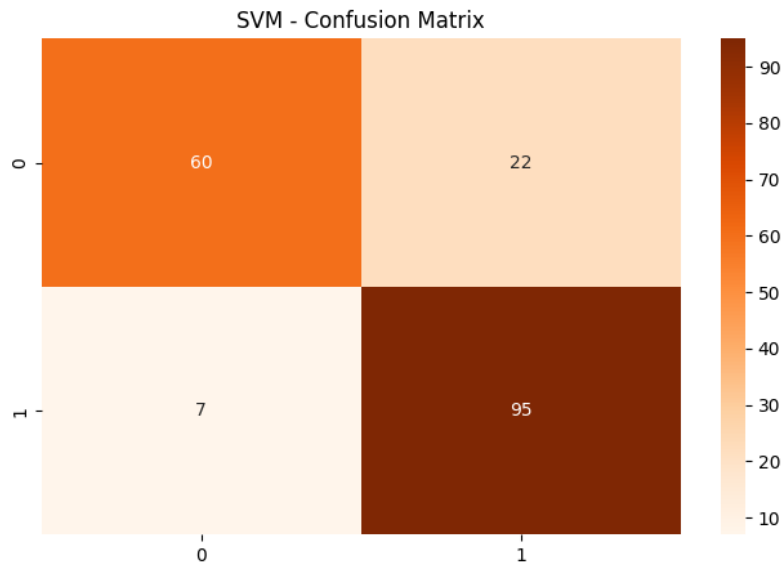


Figura 11: Matrice di confusione - SVM

6.4 Confronto tra i modelli di baseline

La Tabella 7 riassume le prestazioni dei quattro modelli addestrati (Decision Tree non potato, Decision Tree potato, Neural Network, SVM) sul dataset originale, privo di rumore.

Modello	Accuracy	Precision	Recall	F1-score
Decision Tree (baseline)	0.755	0.75	0.75	0.75
Decision Tree (potato)	0.804	0.80	0.80	0.80
Neural Network	0.832	0.83	0.83	0.83
SVM	0.842	0.85	0.84	0.84

Tabella 7: Confronto delle metriche sul test set tra i modelli di baseline (valori pesati/medi).

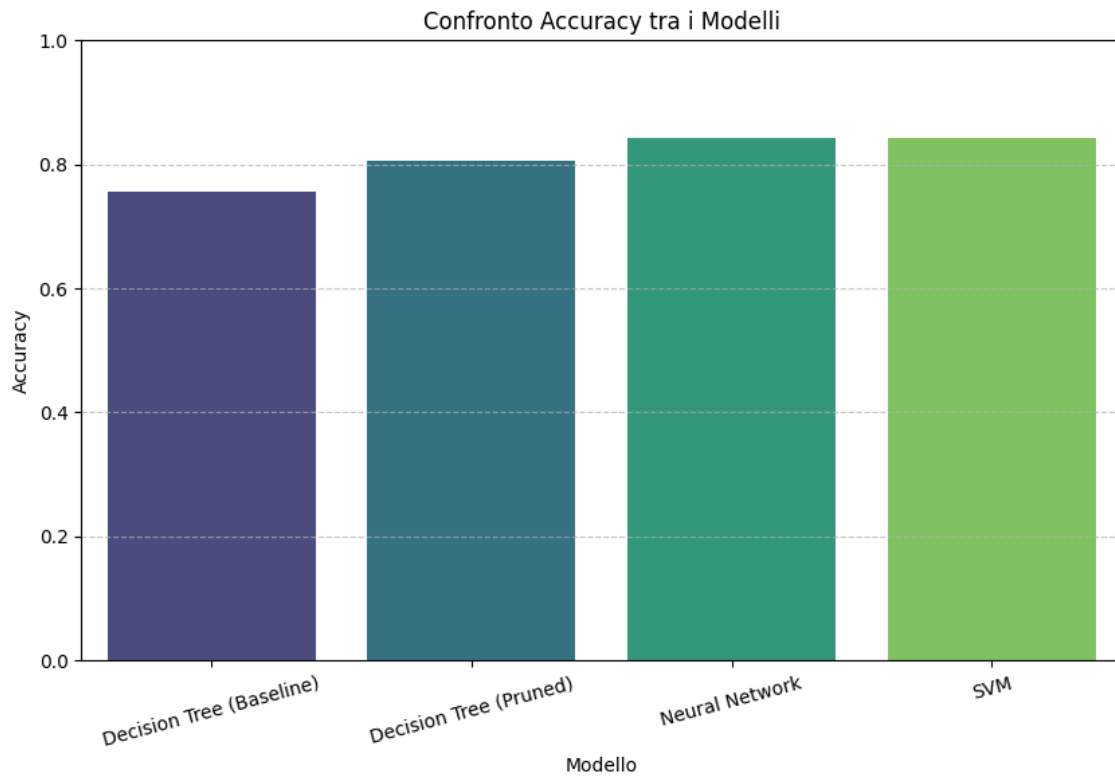


Figura 12: Confronto dell'Accuracy tra i modelli di baseline

La SVM con kernel RBF risulta il modello con le prestazioni migliori sul test set, seguita dalla rete neurale; il Decision Tree potato mostra un netto miglioramento rispetto alla versione non potata, a conferma dell'efficacia del pruning nel ridurre l'overfitting.

7 Introduzione del rumore nel dataset

Per valutare la robustezza dei modelli, è stato introdotto rumore artificiale nel training set (il test set rimane sempre pulito, per garantire un confronto equo tra le diverse condizioni), a tre livelli di intensità crescente:

Livello	Percentuale di rumore
Low	15%
Medium	40%
High	55%

Tabella 8: Livelli di rumore utilizzati negli esperimenti.

Sono state simulate tre diverse tipologie di rumore, ciascuna rappresentativa di problematiche comuni nei dataset reali.

7.1 Valori mancanti (NaN)

È stata implementata una funzione che introduce valori NaN in modo casuale nel training set, generando per ogni cella un numero casuale e sostituendo il valore originale con NaN se tale numero è inferiore alla soglia di rumore desiderata. Il rumore è stato applicato *esclusivamente* su `X_train`, in modo da poter confrontare in modo equo le prestazioni sul medesimo test set pulito. La percentuale effettiva di valori mancanti introdotti è risultata coerente con quella target (15.2%, 40.4%, 55.7% rispettivamente).

7.2 Valori duplicati

È stata implementata una funzione che duplica una percentuale di righe del training set, scelte casualmente (con reinserimento), simulando errori di raccolta/unione dati che generano osservazioni ripetute. Le dimensioni del training set risultante sono riportate in Tabella 9.

Livello	Dimensione training set	Dimensione originale
Low (15%)	846	736
Medium (40%)	1030	736
High (55%)	1140	736

Tabella 9: Dimensione del training set dopo l'inserimento di duplicati.

7.3 Dati "sporchi"

È stata implementata una funzione che introduce rumore realistico differenziato in base al tipo di variabile:

- sulle colonne **continue**, viene aggiunto rumore gaussiano proporzionale alla deviazione standard della colonna stessa, applicato solo a una frazione casuale di righe (in base al livello di rumore);

- sulle colonne **binarie** (incluse quelle ottenute da one-hot encoding), viene effettuato uno *swap* dei valori (0 diventa 1 e viceversa) su una frazione casuale di righe.

Questa tipologia di rumore simula errori di misurazione/inserimento tipici dei dati reali, più insidiosi dei semplici valori mancanti perché non facilmente identificabili.

8 Impatto del rumore sulle prestazioni dei modelli

Per ciascuna tipologia di rumore, i tre modelli (Decision Tree, SVM, Neural Network) sono stati riaddestrati sul training set corrotto e valutati sul medesimo test set pulito, applicando un preprocessing di base (imputazione con mediana per i NaN, scaling standard). Le prestazioni sono state inoltre validate tramite 5-fold cross-validation stratificata (`StratifiedKFold`, $k=5$) sul training set corrotto.

8.1 Valori mancanti

La prima tipologia di rumore che viene tratta è il rumore dovuto ai valori nulli. Per eseguire i test si è modificato il dataset introducendo a scaglioni i valori nulli con le percentuali indicate nel precedente capitolo. Per ogni scaglione è inclusa la versione baseline 0% (il dataframe sprovvisto da rumore) vengono addestrati i tre algoritmi di Machine Learning.

Dopo di che gli algoritmi vengono addestrati, e vengono prodotte le seguenti informazioni, la matrice di confusione, i valori dell'Accuracy, Precision, Recall e F1-Score e un grafico sul accuracy. Tali informazioni ci permettono di capire il comportamento del algoritmo per ogni specifico caso.

Successivamente eseguiamo anche la cross-validation k-fold, con $k = 5$ il quale ci permette di distinguere le prestazioni da un caso fortunato o sfortunato, dandoci dei valori più generali del comportamento degli algoritmi, non solo per un specifico caso.

Di seguito si osservano le matrici di confusione:

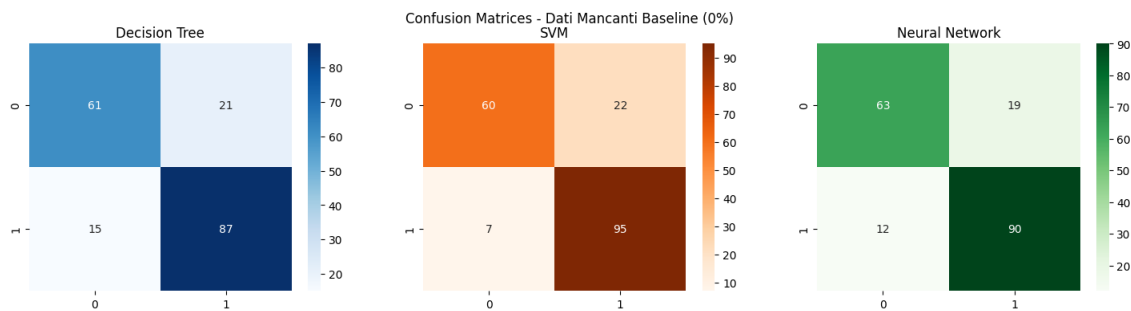


Figura 13: Matrice di confusione della baseline

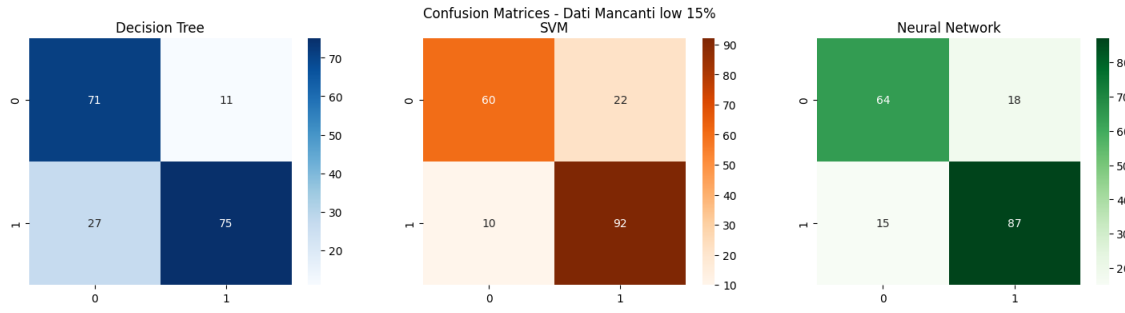


Figura 14: Matrice di confusione con l'introduzione del 15% di valori nulli

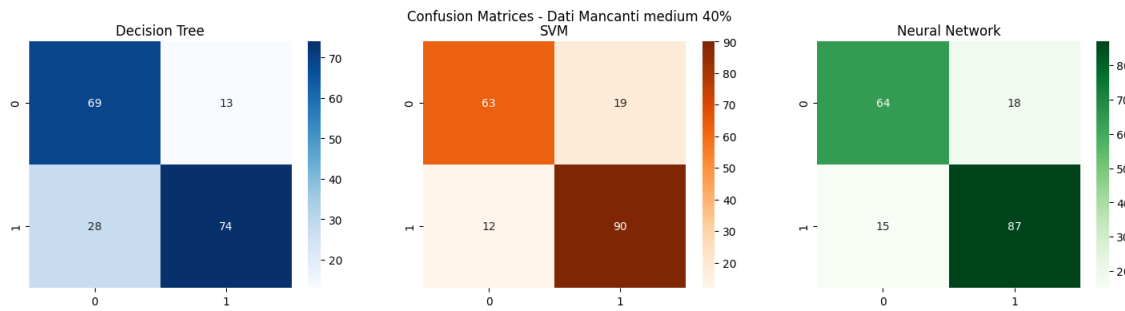


Figura 15: Matrice di confusione con l'introduzione del 40% di valori nulli

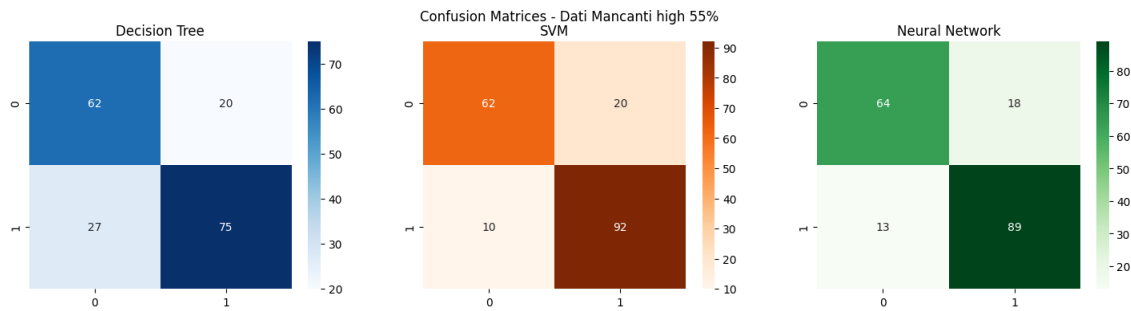


Figura 16: Matrice di confusione con l'introduzione del 55% di valori nulli

Dalle matrici di confusione mostrate nelle Figure 13, 14, 15 e 16, si osservano sulla diagonale principale i valori predetti correttamente (rispettivamente, dall'alto verso il basso, i Veri Negativi e i Veri Positivi), mentre al di fuori di essa (in alto a destra e in basso a sinistra) figurano i valori classificati in modo errato, ovvero i Falsi Positivi e i Falsi Negativi.

La Figura 13 riporta i risultati sul dataset originale (pulito), che rappresenta la baseline di riferimento per valutare il successivo degrado delle performance. In questo scenario si nota una distribuzione ottimale, con un'elevata concentrazione di valori sulla diagonale; la completezza dei dati permette di minimizzare i Falsi Negativi, garantendo un'elevata affidabilità clinica. È già possibile notare come i modelli SVM e Rete Neurale mostrino una precisione superiore rispetto al Decision Tree.

Con l'introduzione progressiva del 15% e del 40% di rumore (Figure 14 e 15), le prestazioni della SVM e della Rete Neurale rimangono pressoché stabili rispetto alla baseline. Al contrario, il Decision Tree inizia a registrare un aumento dei Falsi Negativi. Questo indica che la ricostruzione dei dati tramite imputazione, per quanto efficace, introduce un rumore sufficiente a distorcere i confini decisionali gerarchici tipici di questo modello.

Infine, nello scenario più critico in cui la percentuale di valori nulli supera quella dei dati reali (55%, mostrato in Figura 16), si osserva un netto divario tra le architetture:

- Il Decision Tree mostra il comportamento peggiore, registrando ben 27 Falsi Negativi, un dato che in un contesto clinico è preoccupante. La notevole perdita di informazioni compromette in maniera irreversibile i criteri di split nodale, impedendo all'algoritmo di individuare un numero elevato di pazienti positivi.
- La SVM si dimostra invece l'algoritmo in assoluto più resiliente, limitando i Falsi Negativi a soli 10 casi e mantenendo ben 92 Veri Positivi (un valore vicinissimo ai 95 della baseline). Ciò conferma la capacità del kernel RBF di garantire un iperpiano di separazione estremamente solido.
- La Rete Neurale, infine, si conferma la soluzione più bilanciata: offrendo ottime prestazioni con soli 13 Falsi Negativi, dimostra che un apprendimento basato sull'aggiornamento dei pesi sinaptici e sulle funzioni di attivazione è in grado di tollerare efficacemente il rumore introdotto dall'imputazione semplice.

Livello	Modello	Accuracy	Precision	Recall	F1-Score
<i>Baseline</i>	Decision Tree	0.804	0.806	0.853	0.829
	SVM	0.842	0.812	0.931	0.868
	Neural Network	0.832	0.826	0.882	0.853
<i>Low 15%</i>	Decision Tree	0.793	0.872	0.735	0.798
	SVM	0.826	0.807	0.902	0.852
	Neural Network	0.821	0.829	0.853	0.841
<i>Med 40%</i>	Decision Tree	0.777	0.851	0.725	0.783
	SVM	0.832	0.826	0.882	0.853
	Neural Network	0.821	0.829	0.853	0.841
<i>High 55%</i>	Decision Tree	0.745	0.789	0.735	0.761
	SVM	0.837	0.821	0.902	0.860
	Neural Network	0.832	0.832	0.873	0.852

Tabella 10: Metriche di performance dei modelli su diversi livelli di rumore

La Tabella 10 fornisce un quadro quantitativo e comparativo del degrado prestazionale dei tre modelli all'aumentare dei dati mancanti. Analizzando le metriche di Accuracy, Precision, Recall e F1-Score, emergono dinamiche comportamentali nettamente distinte tra le architetture testate.

In primo luogo, si rileva l'estrema vulnerabilità del Decision Tree. Partendo da una Baseline già inferiore agli altri modelli (Accuracy 0.804), l'algoritmo subisce un crollo progressivo e significativo, scendendo a un'Accuracy di 0.745 al livello di rumore estremo (55%). Il dato più critico riguarda la sua Recall, che precipita da 0.853 a 0.735 già con solo il 15% di rumore. Questo indica che, in presenza di dati imputati, l'albero di decisione diventa un modello "iper-cauto", aumentando temporaneamente la Precision (0.872 al 15%) a scapito della capacità di individuare correttamente i casi positivi (Falsi Negativi), un difetto grave in ambito diagnostico.

Al contrario, la Support Vector Machine (SVM) si conferma come il modello più robusto dell'intero esperimento. Mantiene il primato in tutte le fasi di test: la sua Accuracy oscilla in modo marginale (da 0.842 della Baseline a 0.837 nel livello High), dimostrando una quasi totale indifferenza al rumore. Ancora più rilevante è il mantenimento di una Recall costantemente superiore a 0.88 (raggiungendo 0.902 al 55%), confermando che l'uso del kernel RBF permette di preservare la corretta classificazione dei pazienti malati anche quando oltre metà delle feature viene ricostruita artificialmente.

Infine, la Neural Network evidenzia un comportamento molto stabile e bilanciato. Partendo da ottimi valori iniziali (Accuracy 0.832 e Recall 0.882), la rete gestisce il degrado informativo con notevole resilienza, stabilizzandosi al livello High (55%) su un F1-Score di 0.852, praticamente identico alla Baseline (0.853). Ciò dimostra che la natura distribuita dei pesi sinaptici compensa efficacemente i valori mediani introdotte dal processo di imputazione.

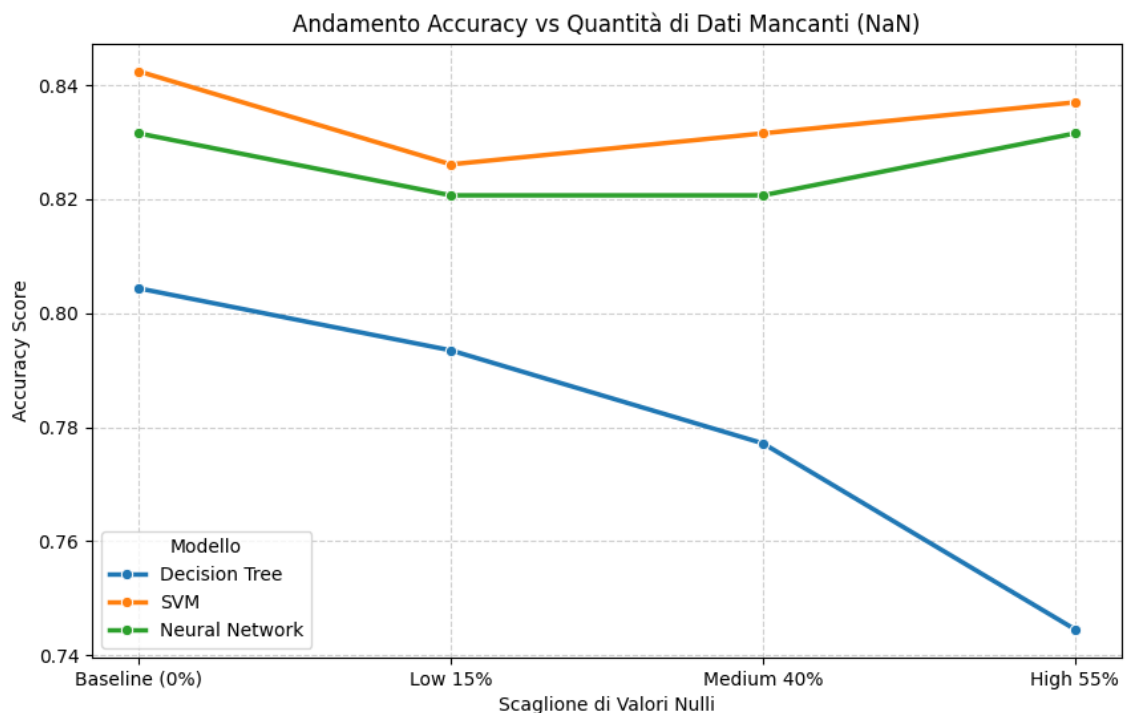


Figura 17: Il grafico mostra l'andamento dell'accuracy al aumentare dei valori nulli per gli algoritmi testati Decision Tree, SVM e Reti Neurali

A conferma di quanto detto finora, osserviamo nella figura 17 come la curva che rappresenta il decision tree ha un netto crollo di accuratezza all'aumentare dei valori

nulli, maggiormente marcato nel momento in cui si superamento del 40%. Mentre i metodi SVM e Reti Neurali hanno un comportamento simile, dove la curva del SVM parte dalla accuratezza più elevata dei tre e tende ad essere piatta nonostante un piccolo calo iniziale tra il 0% e il 15%. In fine, La curva della Rete Neurale si mostra stabile e bilanciata.

8.1.1 Cross-Validation

Per poter validare i risultati analizzati fin ora, e quindi confermare la validità statistica dei valori e assicurare che le performance dei modelli non siano dipendenti da una specifica e fortuita partizione del dataset (train-test split), è stata applicata una tecnica di K-Fold Cross-Validation (con $k = 5$).

Analizzando i risultati medi dell'accuracy nei diversi fold, si ha la prova concreta di quanto un modello sia solido o meno.

La Tabella 11 riporta l'accuracy media di cross-validation al variare del livello di valori mancanti, con preprocessing di base (imputazione mediana + scaling).

Livello NaN	Decision Tree	SVM	Neural Network
Baseline (0%)	0.7541	0.8180	0.8071
Low 15%	0.7296	0.7976	0.7962
Medium 40%	0.7038	0.7609	0.7731
High 55%	0.7161	0.6930	0.7405

Tabella 11: Confronto dell'Accuracy Media CV al variare della percentuale di valori mancanti (Strategia: Base SimpleImputer)

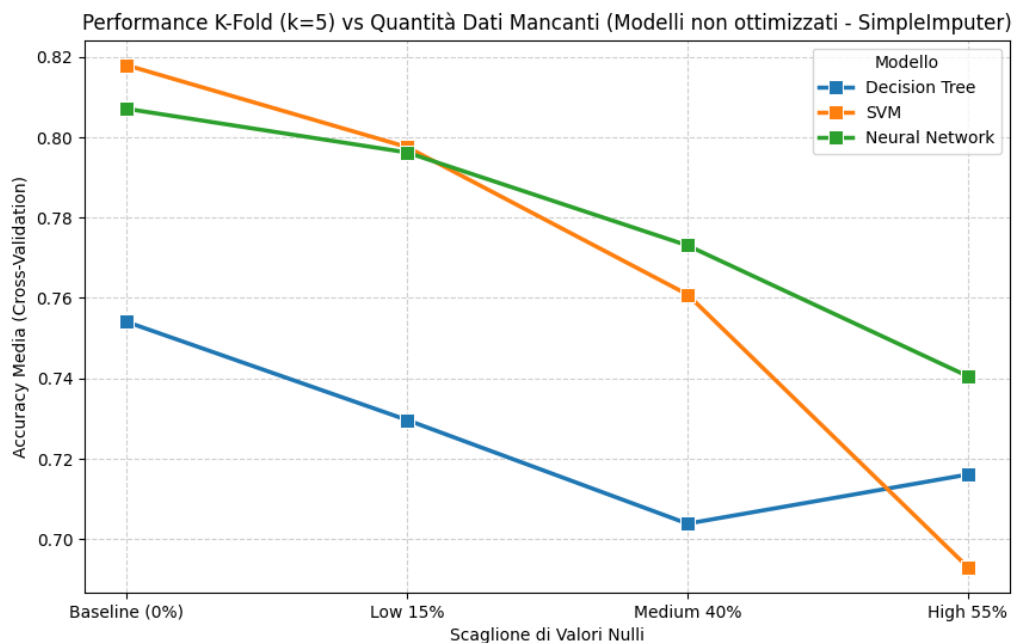


Figura 18: Il grafico mostra l'andamento dell'accuracy eseguendo la cross validation k-fold, $k = 5$, con gli algoritmi non ottimizzati e con l'imputer basico

La K-Fold Cross-Validation a 5 split ribalta i risultati iniziali, svelando la vera vulnerabilità dei modelli al rumore estremo (55%).

La SVM subisce un drastico ridimensionamento: le ottime performance del singolo split erano dovute al caso (high variance). Nella validazione incrociata le prestazioni precipitano, poiché l'uso notevole della mediana azzerava la varianza locale, compromettendo l'iperpiano del kernel RBF. La Neural Network si conferma invece il modello più solido, mantenendo un'Accuracy media superiore a 0.74. A differenza della SVM, l'aggiornamento dei suoi pesi sinaptici tollera efficacemente l'appiattimento informativo causato dall'imputazione semplice.

In conclusione, dal confronto validato tramite Cross-Validation è emersa una chiara gerarchia. Il Decision Tree si è confermato strutturalmente inadeguato a gestire la sparsità informativa, risultando il modello più vulnerabile/fragile. La SVM, inizialmente apparsa come il modello più resiliente, ha rivelato una profonda instabilità geometrica di fronte alla sostituzione dei valori nulli con la mediana della rispettiva colonna, specialmente negli scenari ad alto rumore (55%). Il modello che ha ottenuto la miglior performance in questo test di robustezza è risultato essere la Neural Network. In assenza di ottimizzazioni avanzate e di imputazioni multivariate, il modello neurale ha offerto la maggiore affidabilità predittiva, assorbendo le anomalie dei dati ricostruiti grazie alla natura distribuita delle sue funzioni di attivazione.

8.2 Valori duplicati

Analogamente alle altre tipologie di rumore, i tre modelli sono stati riaddestrati sul training set con duplicati e valutati sul medesimo test set pulito. Le matrici di confusione per i quattro livelli sono riportate nelle Figure 19–22.

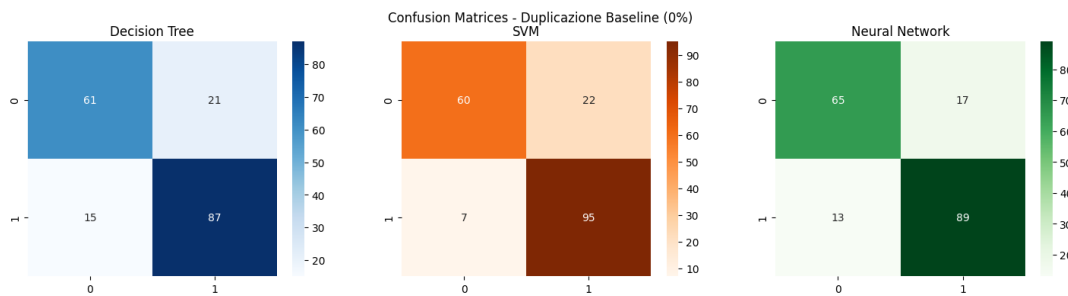


Figura 19: Matrice di confusione con il training set originale (Baseline, 0% duplicati)

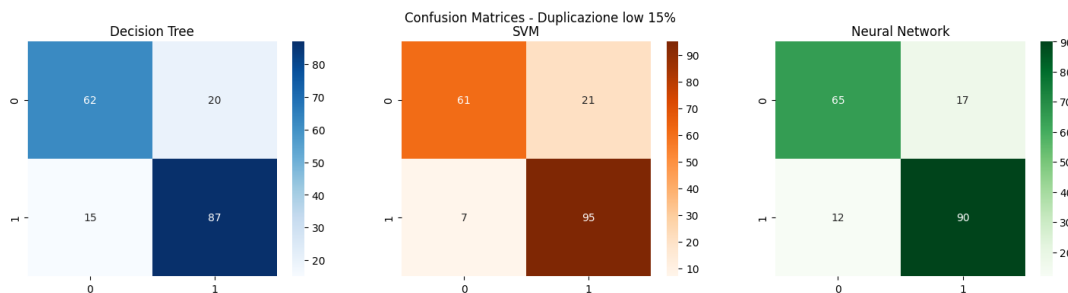


Figura 20: Matrice di confusione con il 15% di righe duplicate

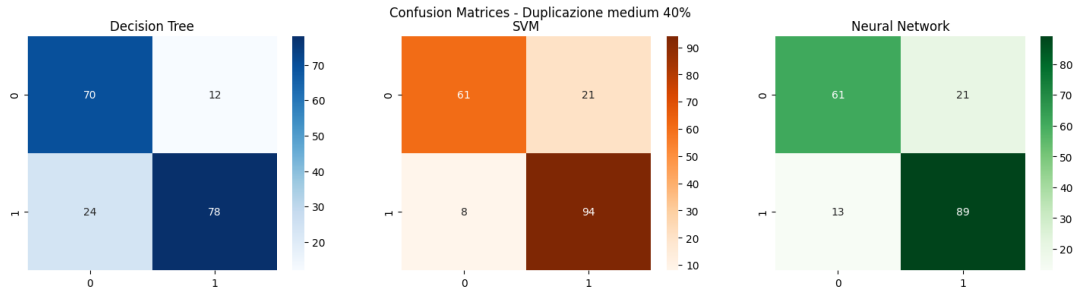


Figura 21: Matrice di confusione con il 40% di righe duplicate

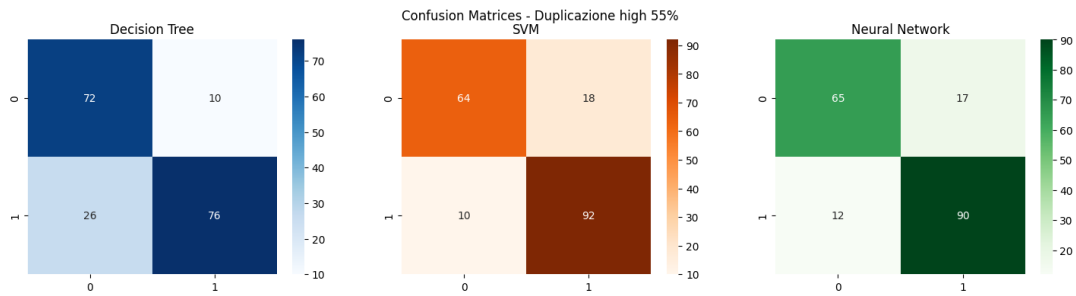


Figura 22: Matrice di confusione con il 55% di righe duplicate

A differenza di quanto osservato per i valori mancanti e per i dati sporchi, la presenza di righe duplicate produce un effetto pressoché trascurabile sul test set pulito. Il Decision Tree mantiene 15 Falsi Negativi tra Baseline e Low, per poi salire a 24 e 26 rispettivamente ai livelli Medium e High; la SVM passa da 7 (Baseline/Low) a 8 (Medium) e 10 (High) Falsi Negativi; la Neural Network resta stabile tra 12 e 13 casi in tutti gli scenari. Questo comportamento è coerente con la natura del rumore introdotto: duplicare una riga non aggiunge informazione nuova, ma ne aumenta soltanto il peso relativo nella funzione di perdita durante l'addestramento.

Livello	Modello	Accuracy	Precision	Recall	F1-Score
Baseline (0%)	Decision Tree	0.804	0.806	0.853	0.829
	SVM	0.842	0.812	0.931	0.868
	Neural Network	0.837	0.840	0.873	0.856
Low (15%)	Decision Tree	0.810	0.813	0.853	0.833
	SVM	0.848	0.819	0.931	0.872
	Neural Network	0.842	0.841	0.882	0.861
Medium (40%)	Decision Tree	0.804	0.867	0.765	0.813
	SVM	0.842	0.817	0.922	0.866
	Neural Network	0.815	0.809	0.873	0.840
High (55%)	Decision Tree	0.804	0.884	0.745	0.809
	SVM	0.848	0.836	0.902	0.868
	Neural Network	0.842	0.841	0.882	0.861

Tabella 12: Metriche di performance sul test set, ai vari livelli di duplicazione del training set.

L'unico modello a mostrare un degrado apprezzabile è il Decision Tree, la cui Recall scende da 0.853 (baseline) a 0.745 (High 55%). La SVM mantiene un F1-Score compreso tra 0.866 e 0.872 in tutti gli scenari; la Neural Network oscilla in un intervallo leggermente più ampio (0.840–0.861), con un minimo al livello Medium, senza però mostrare alcun trend di degrado sistematico.

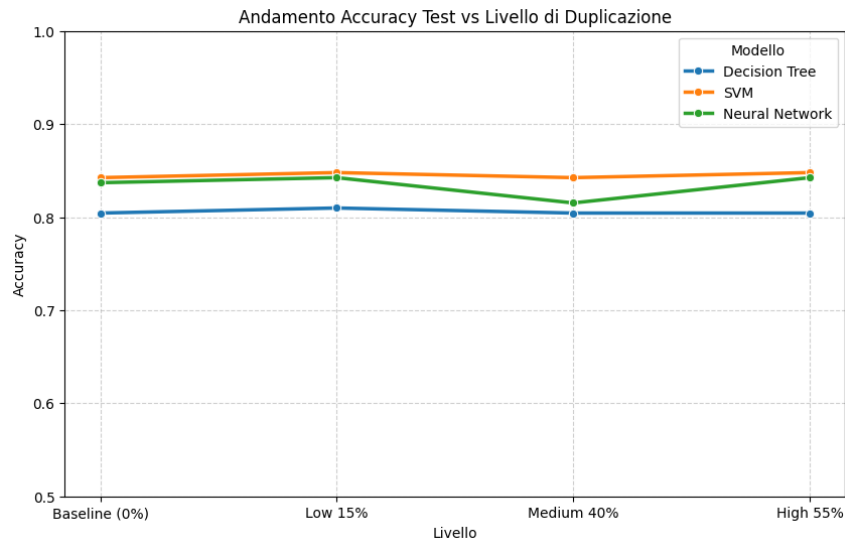


Figura 23: Andamento dell'Accuracy sul test set al variare del livello di duplicazione

A differenza delle curve nette osservate per i valori mancanti e per i dati sporchi, le curve in Figura 23 restano pressoché piatte, con oscillazioni contenute entro 1–2 punti percentuali. Questo risultato è importante da confrontare con quanto emerge dalla Cross-Validation nella sotto-sezione seguente.

8.2.1 Cross-Validation

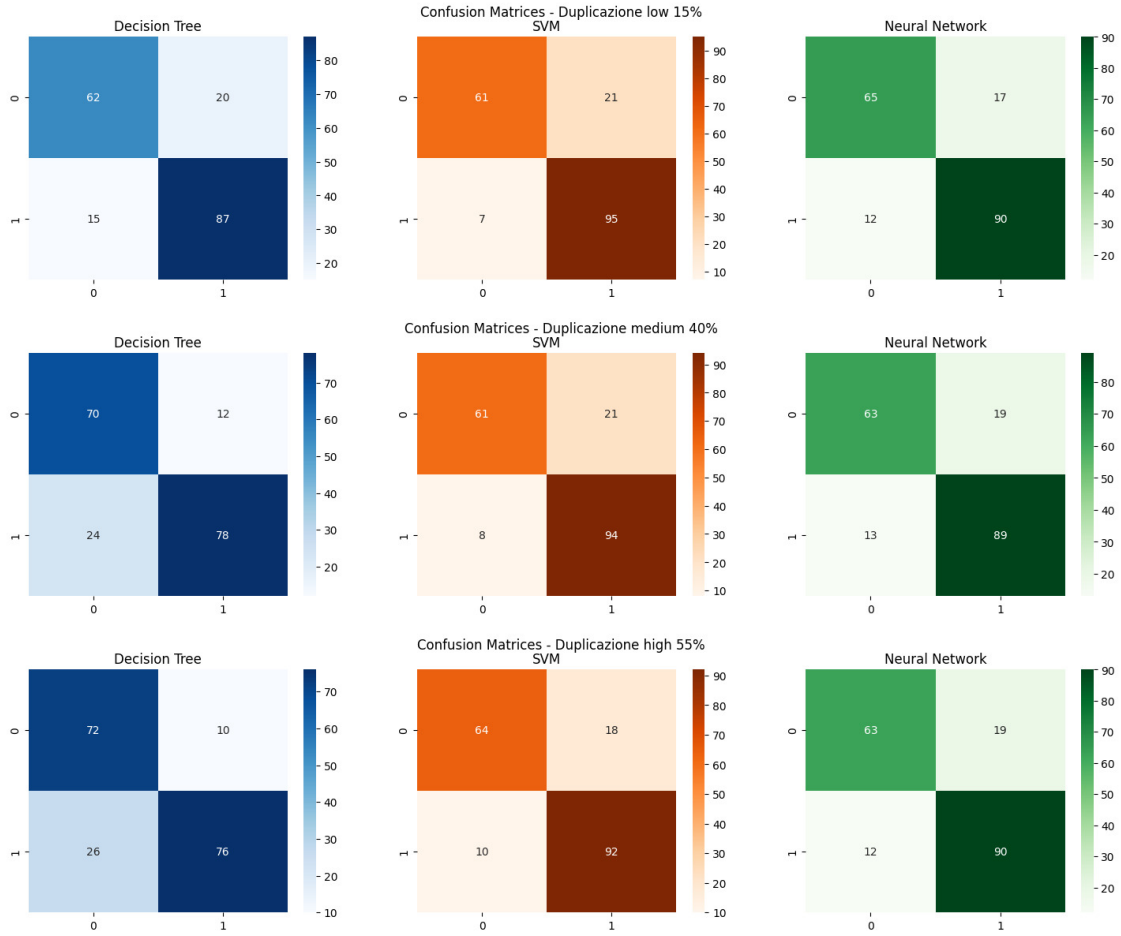


Figura 24: Confronto dell'Accuracy tra i modelli di baseline

Livello	Decision Tree	SVM	Neural Network
Baseline (0%)	0.754	0.818	0.807
Low (15%)	0.784	0.830	0.811
Medium (40%)	0.777	0.847	0.824
High (55%)	0.799	0.854	0.823

Tabella 13: Accuracy media di CV (5-fold) al variare del livello di duplicazione (dati non ripuliti).

A differenza delle altre tipologie di rumore, l'accuracy in cross-validation **aumenta** apparentemente con il livello di duplicazione. Questo risultato è fuorviante: è dovuto a un fenomeno di *data leakage*, poiché la suddivisione in fold della cross-validation può separare copie identiche della stessa riga tra training e validation del fold, rendendo la stima della performance eccessivamente ottimistica.

Il confronto diretto tra la Tabella 12 (valutazione onesta sul test set) e la Tabella 13 (valutazione in Cross-Validation) permette di isolare due dinamiche:

- **Il segnale reale è piatto:** sul test set indipendente l'Accuracy resta sostanzialmente invariata al crescere della duplicazione per tutti e tre i modelli, a conferma che duplicare righe non introduce né informazione utile né rumore dannoso;
- **Il segnale in CV è un artefatto, non legato alla complessità del modello:** tra Baseline e High 55% l'accuracy di CV cresce di +4.5 punti percentuali per il Decision Tree (0.754→0.799), +3.6 per la SVM (0.818→0.854) e +1.6 per la Neural Network (0.807→0.823). Nessuno di questi incrementi trova riscontro in Tabella 12: il fatto che sia interessato anche un modello semplice e regolarizzato come il Decision Tree dimostra che il leakage è una proprietà dello split dei dati, non dell'algoritmo, e va quindi corretto a monte (Sezione 9.2), non tramite tecniche di regolarizzazione del modello.

8.3 Dati sporchi

In questa sezione analizziamo il comportamento dei 3 modelli (SVM, Decision Tree, e Reti Neurali) con l'introduzione di valori sporchi sempre come visto nei precedenti casi a scaglioni. In questa fase di test introducendo valori sbagliati il rischio per gli algoritmi è quello di apprendere pattern errati.

Di seguito, analizziamo le matrici di confusione per ogni livello di valori sporchi aggiunti, per analizzare il livello di degrado che viene raggiunto dagli algoritmi e quale di esso abbia il comportamento migliore.

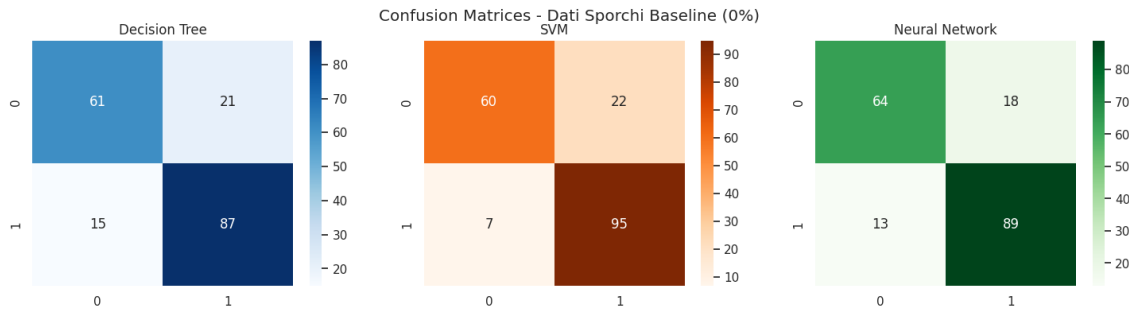


Figura 25: Matrice di confusione per i tre modelli di ML senza valori sporchi aggiunti

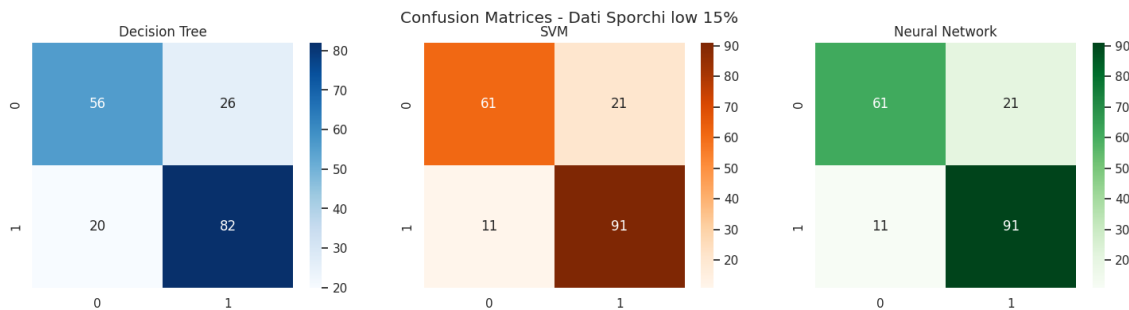


Figura 26: Matrice di confusione per i tre modelli di ML con il 15% di valori sporchi aggiunti

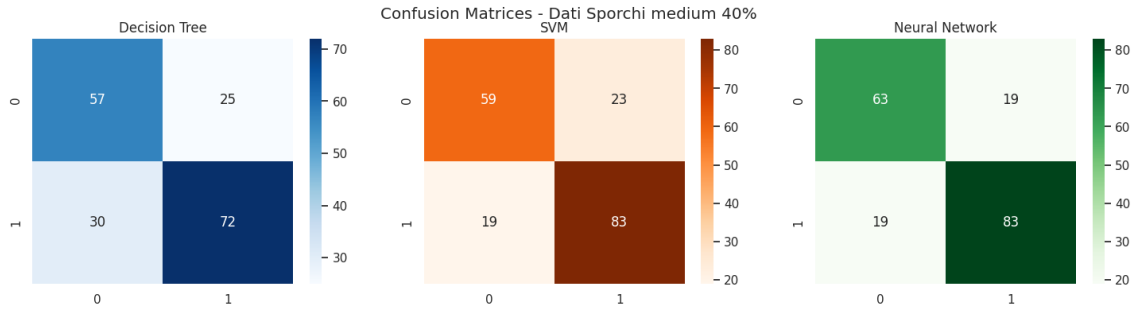


Figura 27: Matrice di confusione per i tre modelli di ML con il 40% di valori sporchi aggiunti

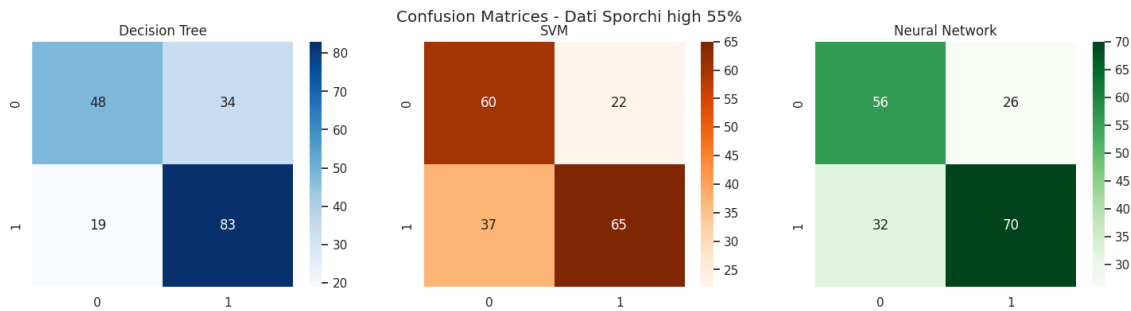


Figura 28: Matrice di confusione per i tre modelli di ML con il 55% di valori sporchi aggiunti

La Figura 25 mostra il comportamento degli algoritmi con il dataset pulito; questa serie di matrici rappresenta la baseline di riferimento che utilizzeremo per valutare il livello di degrado al crescere della percentuale di valori sporchi (rumore) inseriti nel dataset.

Con l'introduzione del 15% di valori sporchi, come mostrato nella Figura 26, si può notare che la Rete Neurale e la SVM dimostrano un'ottima resilienza iniziale, presentando matrici di confusione pressoché identiche a quelle della baseline. Al contrario, il Decision Tree mostra già un primo calo delle prestazioni, passando da 15 a 20 Falsi Negativi.

Aumentando la percentuale di dati sporchi al 40% (Figura 27), il Decision Tree evidenzia forti difficoltà nella classificazione, raggiungendo i 30 Falsi Negativi. I modelli SVM e Rete Neurale, seppur comportandosi decisamente meglio rispetto all'albero di decisione, subiscono un raddoppio delle classificazioni errate di tipo Falso Negativo. Questo andamento conferma che, a questo livello di corruzione, l'addestramento inizia a interiorizzare i falsi pattern causati dai valori sporchi, i quali distorcono lo spazio vettoriale e l'aggiornamento dei pesi sinaptici.

Infine, raggiungendo il 55% di valori sporchi (superando quindi la quantità di dati reali), si osserva un drastico decadimento prestazionale, come visibile nella Figura 28. La SVM subisce un ulteriore raddoppio dei Falsi Negativi, dimostrando che il rumore ha alterato le distanze vettoriali a tal punto da rendere il kernel RBF incapace

di tracciare un margine di separazione sensato, portando a misclassificare oltre un terzo dei pazienti malati. Anche la Rete Neurale subisce un grave peggioramento, toccando i 32 Falsi Negativi: l'eccesso di rumore satura i gradienti durante l'addestramento, impedendo al modello di distinguere il segnale clinico dalle anomalie. Il Decision Tree, invece, registra un comportamento anomalo: i Falsi Negativi si riducono a 19, ma si assiste a un'esplosione dei Falsi Positivi. Questo squilibrio è riconducibile alla generazione di regole di split caotiche che, nel tentativo di mitigare l'elevata incertezza dei valori, finiscono per favorire la classe positiva in modo fuorviante.

Livello	Modello	Accuracy	Precision	Recall	F1-Score
<i>Baseline (0%)</i>	Decision Tree	0.804	0.806	0.853	0.829
	SVM	0.842	0.812	0.931	0.868
	Neural Network	0.832	0.832	0.873	0.852
<i>Low 15%</i>	Decision Tree	0.750	0.759	0.804	0.781
	SVM	0.826	0.812	0.892	0.850
	Neural Network	0.826	0.812	0.892	0.850
<i>Medium 40%</i>	Decision Tree	0.701	0.742	0.706	0.724
	SVM	0.772	0.783	0.814	0.798
	Neural Network	0.793	0.814	0.814	0.814
<i>High 55%</i>	Decision Tree	0.712	0.709	0.814	0.758
	SVM	0.679	0.747	0.637	0.688
	Neural Network	0.685	0.729	0.686	0.707

Tabella 14: Metriche di performance dei modelli ai vari livelli di corruzione del dataset (Dati Sporchi).

La Tabella 14 quantifica il degrado causato dall'inserimento di dati sporchi nel set di addestramento, evidenziando due principalmente due situazioni:

- La SVM e la Rete Neurale nonostante sono stabili al 15% di rumore (F1-Score 0.850), le prestazioni precipitano al 55%. È particolarmente critico il peggioramento della Recall per la SVM (da 0.931 a 0.637): a causa dei falsi pattern, il modello perde quasi il 30% della sua sensibilità diagnostica.
- Il Decision Tree come visto prima Al 55% di rumore mostra un'anomalia dove l'albero registra una Recall elevata (0.814) accompagnata però dalla peggior Precision del test (0.709). Per gestire i dati sporchi, l'algoritmo sovra-prevede sistematicamente la classe positiva, generando un eccesso di Falsi Positivi che ne azzerà l'affidabilità clinica.

Le dinamiche finora descritte trovano perfetta corrispondenza grafica nella Figura 29. È evidente il crollo verticale delle prestazioni per SVM e Rete Neurale non appena il rumore supera il 15%. Risulta inoltre ben visibile l'anomala risalita della curva del Decision Tree nello scenario più estremo (55%): come dimostrato dai dati tabellari (tabella 14, questo 'miglioramento' dell'Accuracy è falso, frutto di uno sbilanciamento predittivo che non garantisce alcuna reale affidabilità clinica.

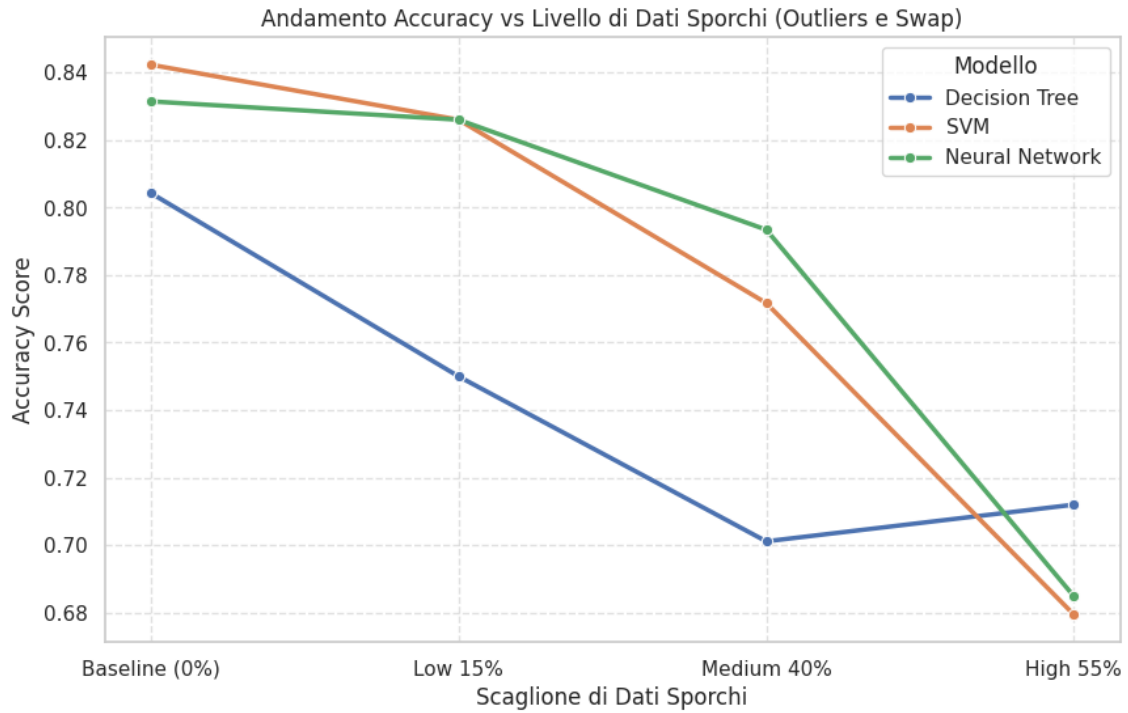


Figura 29: Grafico sul accuratezza al aumentare dei valori sporchi

8.3.1 Cross Validation

A questo punto per avere la certezza che l'analisi sia solida e che i risultati non siano frutto di un singolo train-test split particolarmente sbilanciato, si applica la K-Fold Cross-Validation a 5 split. Guardando le medie dell'Accuracy, emerge chiaramente quanto i modelli base siano vulnerabili di fronte all'inserimento di dati falsati. I numeri sono riassunti nella tabella 15 sottostante:

Livello	Decision Tree	SVM	Neural Network
Baseline (0%)	0.754	0.818	0.808
Low 15%	0.731	0.766	0.761
Medium 40%	0.693	0.683	0.694
High 55%	0.647	0.697	0.702

Tabella 15: Accuracy media in K-Fold Cross-Validation (K=5) per ciascun modello e livello di corruzione.

Di seguito vengono mostrati i dati della tabella 15 sotto forma grafica.

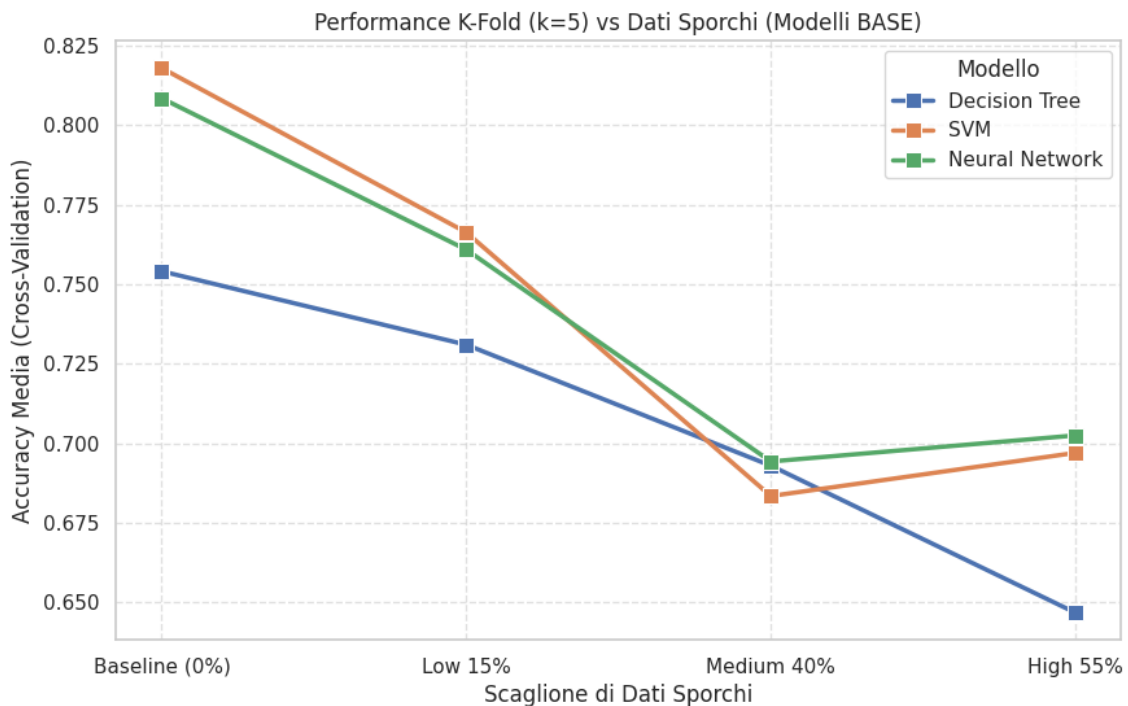


Figura 30: Andamento della Accuracy per ciascun modello tramite Cross-Validation k-fold

L'analisi di queste misurazioni permette di evidenziare tre dinamiche fondamentali:

- Il risultato più importante della Cross-Validation si ottiene osservando l'andamento dell'albero di decisione, dove il finto miglioramento notato nel singolo test al 55% scompare del tutto. Calcolando la media sui 5 fold, l'Accuracy precipita al valore più basso dell'intero esperimento (0.647). Questo dimostra che l'algoritmo non sa adattarsi al rumore, ma si limita a creare regole di classificazione instabili e prive di logica.
- A differenza di quanto visto con i dati mancanti, l'inserimento di dati "sporchi" ha un effetto distruttivo molto più rapido. Già al 40% di rumore, tutti i modelli crollano sotto la soglia dello 0.70. Questo accade perché gli algoritmi apprendono dei pattern falsi, i quali causano un forte disturbo nella fase di addestramento, alterando i gradienti (per la Rete Neurale) e i vettori di supporto (per la SVM).
- Nello scenario peggiore, dove è presente il 55% di valori corrotti, la Rete Neurale (0.702) e la SVM (0.697) ottengono risultati quasi identici, fermandosi su livelli di affidabilità del tutto inaccettabili. Tuttavia, la Rete Neurale mantiene un minimo vantaggio rispetto agli altri modelli.

In conclusione, questo test dimostra una regola fondamentale: nessun modello, per quanto avanzato, può formulare una predizione corretta se più della metà delle informazioni che gli vengono fornite è corrotta. Pertanto, questa tipologia di rumore risulta la più dannosa per le prestazioni. Tutti e tre i modelli mostrano un calo consistente e progressivo dell'Accuracy all'aumentare del livello di rumore, poiché i valori errati alterano direttamente la relazione logica tra feature e target senza essere facilmente rilevabili e isolabili, come avviene invece nel caso dei valori nulli.

9 Tecniche di preprocessing e regolarizzazione per la mitigazione del rumore

Per ciascuna tipologia di rumore sono state applicate tecniche specifiche di preprocessing e/o regolarizzazione dei modelli, al fine di valutarne l'efficacia nel contenere il degrado delle prestazioni osservato nella sezione precedente.

9.1 Mitigazione dei valori mancanti

Oltre all'imputazione di base con la mediana, sono state testate due strategie di imputazione più avanzate, in combinazione con tecniche di regolarizzazione dei modelli (Decision Tree con `max_depth=4`, `min_samples_split=20`, `min_samples_leaf=15`; SVM con margine più "morbido", $C = 0.1$; rete neurale con Dropout (0.3 e 0.2) e EarlyStopping):

- **KNN Imputer**: imputa ciascun valore mancante sulla base della media dei $k = 5$ vicini più simili;
- **Iterative Imputer**: stima ogni valore mancante modellandolo come funzione delle altre variabili, in modo iterativo (`max_iter=10`).

La Tabella 16 confronta l'accuracy media di cross-validation ottenuta con le tre strategie di imputazione (per la SVM, unico modello riportato a titolo di confronto sintetico – i risultati completi per tutti i modelli sono disponibili nel notebook).

Livello	SimpleImputer (base)	KNN Imputer	Iterative Imputer
Baseline (0%)	0.818	0.823	0.823
Low (15%)	0.798	0.799	0.795
Medium (40%)	0.761	0.768	0.766
High (55%)	0.693	0.723	0.739

Tabella 16: Accuracy media di CV (SVM) al variare della strategia di imputazione dei valori mancanti.

Le strategie di imputazione avanzata (KNN e Iterative Imputer), unite alla regolarizzazione dei modelli, mostrano un beneficio soprattutto ai livelli di rumore più elevati (Medium e High), dove il recupero di informazione tramite le relazioni tra variabili risulta più efficace della semplice sostituzione con la mediana.

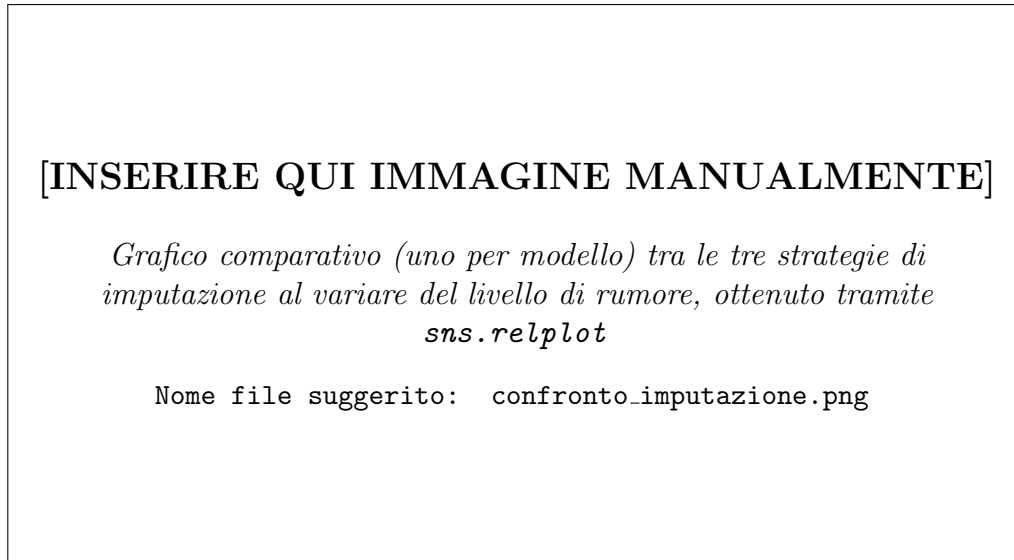


Figura 31: Confronto tra le strategie di imputazione dei valori mancanti (per modello)

9.2 Mitigazione dei valori duplicati

Sono state confrontate due strategie per la gestione dei duplicati:

- **Rimozione dei duplicati:** identificazione ed eliminazione delle righe identiche (`drop_duplicates()`), per evitare che il modello attribuisca peso eccessivo a osservazioni ripetute e per prevenire il *data leakage* in fase di validazione. Applicando questa tecnica, il training set torna, per tutti i livelli, all'esatta dimensione originale di 735 righe (Tabella 17);
- **Aggregazione intelligente:** anziché rimuovere semplicemente le righe duplicate, si raggruppano le osservazioni identiche e si assegna come etichetta la *moda* del target tra le occorrenze duplicate. Questa tecnica risulta particolarmente utile quando i duplicati presentano etichette discordanti (rumore di etichettatura).

Livello	Righe rimosse	Dimensione finale
Baseline (0%)	0	735
Low (15%)	111	735
Medium (40%)	295	735
High (55%)	405	735

Tabella 17: Effetto della rimozione dei duplicati sulla dimensione del training set.

Il grafico seguente confronta l'accuratezza media ottenuta tramite Cross-Validation (5-Fold) sui dati grezzi con duplicati rispetto a quella ottenuta applicando le strategie di pulizia.

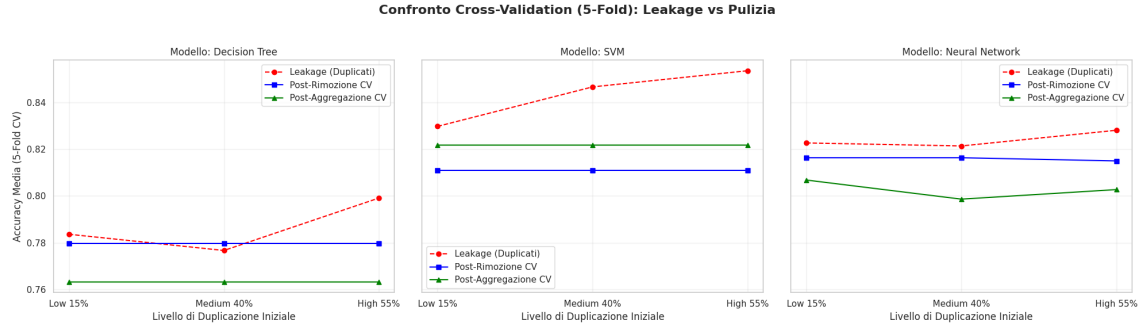


Figura 32: Confronto tra le strategie di gestione dei duplicati in Cross-Validation per i tre modelli analizzati.

Come si evince dalla Figura 32, la valutazione sui dati non trattati (linea tratteggiata rossa) soffre di un evidente fenomeno di *data leakage*. All’aumentare della percentuale di duplicati, l’accuratezza della Cross-Validation aumenta fittiziamente (visibile in particolare nei modelli più complessi come la SVM e le Reti Neurali). Questo accade perché le stesse osservazioni finiscono per essere presenti simultaneamente sia nei fold di addestramento che in quello di validazione, permettendo al modello di riconoscere pattern già visti anziché generalizzare su dati nuovi. Al contrario, le strategie di rimozione e aggregazione restituiscono stime affidabili e non distorte. Poiché entrambe le tecniche riescono a identificare e neutralizzare completamente le righe ridondanti — riconducendo il training set all’esatta composizione della Baseline originale (Tabella 17) — le curve di accuratezza post-regolarizzazione risultano perfettamente piatte e costanti al variare del livello di duplicazione iniziale. Questo andamento dimostra matematicamente l’efficacia della pulizia e la necessità di gestire i duplicati *prima* di qualsiasi fase di validazione.

9.3 Mitigazione dei dati sporchi

Per contrastare l’effetto dei dati ”sporchi” (rumore gaussiano e swap di valori binari), che si sono rivelati la tipologia di rumore più impattante, è stata adottata una pipeline di preprocessing robusto composta da:

- **RobustScaler**: normalizzazione basata su mediana e range interquartile, meno sensibile agli outlier rispetto allo **StandardScaler** classico;
- **PCA (Principal Component Analysis)**, mantenendo il numero minimo di componenti necessario a spiegare il **95%** della varianza totale, con l’obiettivo di filtrare parte del rumore concentrato nelle componenti a bassa varianza;
- modelli regolarizzati (Decision Tree con `max_depth=4`, `min_samples_split=20`, `min_samples_leaf=15`; SVM con $C = 0.1$).

Il numero di componenti principali necessarie per raggiungere il 95% della varianza spiegata, per ciascun livello di rumore, è riportato in Tabella 18.

Livello	Componenti principali (95% varianza)
Baseline (0%)	14
Low (15%)	16
Medium (40%)	14
High (55%)	16

Tabella 18: Numero di componenti principali necessarie per spiegare il 95% della varianza, per livello di rumore.

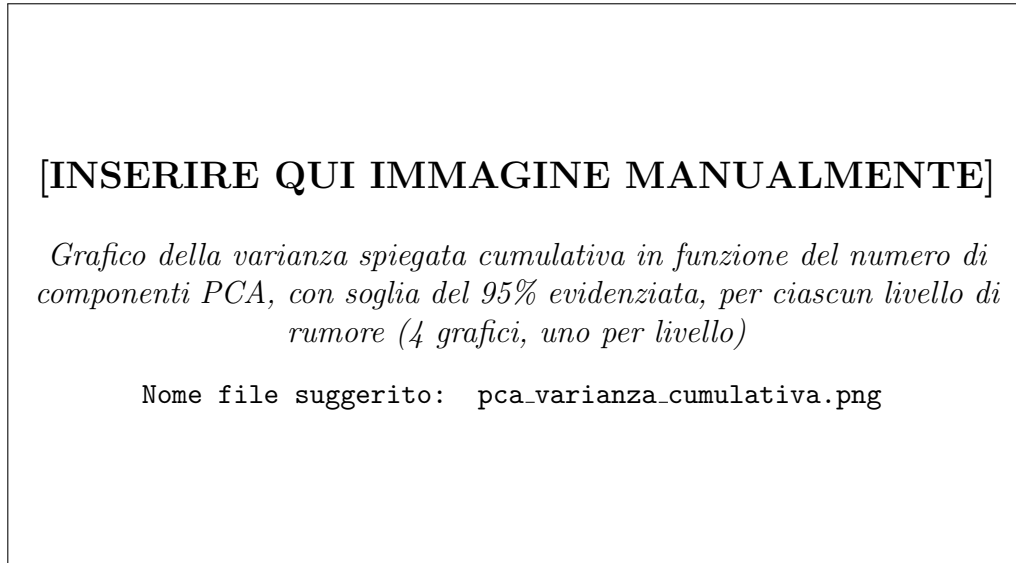


Figura 33: Analisi PCA – Varianza cumulativa spiegata per livello di rumore

La Tabella 19 riporta l'accuracy media di cross-validation ottenuta con la pipeline robusta (RobustScaler + PCA + modelli regolarizzati), da confrontare con i valori di Tabella ?? ottenuti senza tecniche di mitigazione.

Livello	Decision Tree	SVM	Neural Network
Baseline (0%)	0.745	0.811	0.798
Low (15%)	0.726	0.769	0.750
Medium (40%)	0.669	0.698	0.704
High (55%)	0.689	0.708	0.702

Tabella 19: Accuracy media di CV (5-fold) con pipeline robusta (RobustScaler + PCA + regolarizzazione) sui dati sporchi.

Il confronto tra le due tabelle mostra come, per questa specifica tipologia di rumore, il beneficio della pipeline robusta risulti contenuto e non sempre univoco: ai livelli Low e High le prestazioni risultano leggermente superiori rispetto al caso non regolarizzato, mentre al livello Medium risultano sostanzialmente comparabili. Questo suggerisce che il rumore gaussiano applicato direttamente sulle feature, unito allo swap dei valori binari, alteri in modo diffuso la struttura informativa del dataset, rendendo difficile una sua completa mitigazione tramite le sole tecniche di preprocessing lineare (scaling e riduzione dimensionale).

10 Confronto finale tra le strategie

Per fornire una visione d'insieme, i risultati di tutte le strategie di gestione del rumore analizzate sono stati raccolti in un unico confronto grafico, organizzato per tipologia di rumore (valori mancanti con KNN Imputer, valori mancanti con Iterative Imputer, duplicati rimossi, duplicati aggregati, dati sporchi con pipeline robusta) e per modello (Decision Tree, SVM, Neural Network), al variare del livello di rumore.

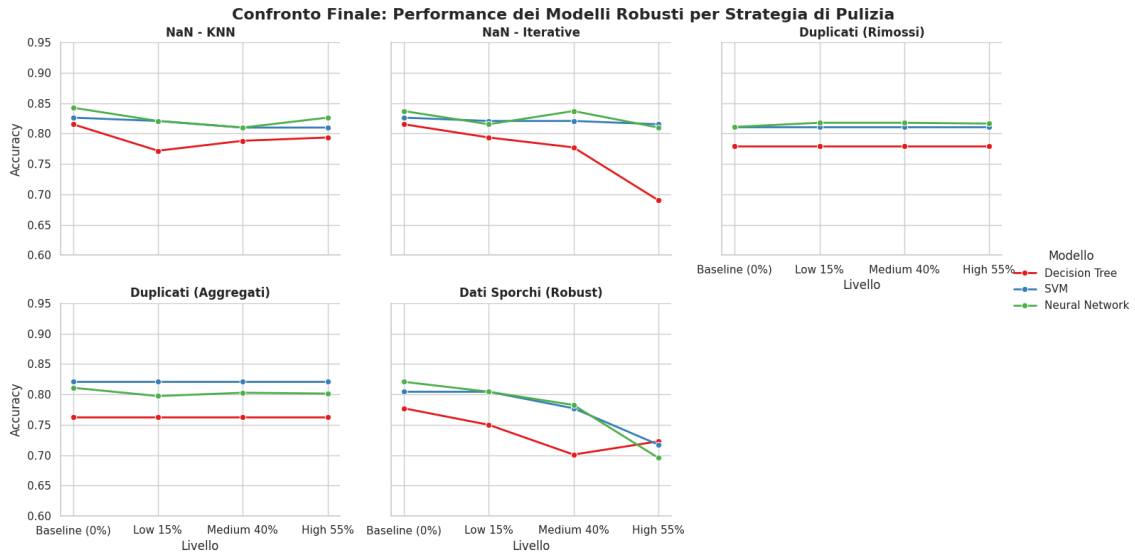


Figura 34: Confronto finale delle prestazioni dei modelli robusti per strategia di pulizia dei dati

Dall'analisi complessiva emergono le seguenti osservazioni:

- la **SVM** risulta, in generale, il modello più performante e più stabile al crescere del rumore, specialmente per i valori mancanti e i dati sporchi;
- i **valori mancanti** sono la tipologia di rumore più facilmente mitigabile, grazie a tecniche di imputazione relativamente mature (KNN, Iterative Imputer);
- i **duplicati**, se non correttamente identificati, non alterano necessariamente il potere predittivo del modello sul test set, ma **falsano sistematicamente la stima** delle prestazioni in fase di addestramento (data leakage); l'applicazione di strategie di rimozione o aggregazione ripristina la correttezza della validazione, garantendo una valutazione delle prestazioni coerente con la baseline originale;
- i **dati sporchi** (rumore gaussiano e inversione di valori binari) costituiscono la sfida più difficile da mitigare, poiché alterano silenziosamente il contenuto informativo delle feature senza essere identificabili come nel caso dei valori mancanti.

11 Conclusioni

Il progetto ha permesso di analizzare in modo sistematico l'impatto di tre diverse tipologie di rumore (valori mancanti, duplicati, dati sporchi) sulle prestazioni di

tre modelli di Machine Learning (Decision Tree, SVM, rete neurale), applicati a un problema reale di classificazione in ambito medicale.

I risultati mostrano che:

1. l'impatto del rumore **non è uniforme**: i valori mancanti causano un degrado moderato gestibile tramite imputazione; i duplicati rappresentano una minaccia subdola poiché, pur non degradando direttamente il modello, **inquinano il processo di validazione**, portando a una sovrastima delle performance; i dati sporchi, infine, costituiscono la sfida più complessa poiché compromettono silenziosamente l'integrità del segnale informativo;
2. le tecniche di **regolarizzazione dei modelli** (pruning per il Decision Tree, soft margin per la SVM, dropout ed early stopping per la rete neurale) contribuiscono a contenere l'overfitting e, in generale, a rendere i modelli più stabili al crescere del rumore;
3. la **cross-validation** si conferma uno strumento fondamentale, ma richiede estrema attenzione: **la presenza di duplicati nel training set agisce come una forma di data leakage**, fornendo stime di accuratezza eccessivamente ottimistiche che non trovano riscontro sul test set;
4. non esiste un'unica tecnica di preprocessing "universale": la scelta della strategia più efficace dipende dalla natura specifica del rumore presente nei dati.

Come possibile sviluppo futuro, si potrebbero esplorare tecniche di rilevamento automatico del tipo di rumore presente nel dataset, così da selezionare in modo adattivo la strategia di preprocessing più adeguata, oltre a estendere l'analisi ad altri modelli (es. ensemble methods come Random Forest o Gradient Boosting) e ad altri dataset di dominio diverso, per verificare la generalizzabilità delle conclusioni ottenute.

In conclusione, il lavoro mostra come la qualità dei dati sia determinante almeno quanto la scelta del modello di Machine Learning: una corretta gestione del rumore può incidere sulle prestazioni tanto quanto la scelta dell'algoritmo.